Auditing Controller Replacement after Environment-Model Change

Anonymous authors

***Abstract*—After an environment-model change, a replacement controller may need to continue from the current state with pending obligations. We audit when a negative translated synthesis answer can justify excluding such a continuation. For the one-way Boolean-helper encoding used in Kind synthesis, we characterize exactly which strict-winning source continuations survive. The helper input can release environment safety one step before controller safety. With guards covering all safety duties, a continuation survives exactly when one winning source strategy keeps a safe response to the helper's first change available for as long as the helper may wait. This yields a necessary and sufficient whole-region preservation condition and an extension for retained safety duties. A constructed Patrol update shows why responding safely now is insufficient: fulfilling a charging obligation can destroy that response option. Separate Vehicle and Obstacle analyses distinguish initialization, continuation preservation and tool answers. At a fixed Obstacle query, releasing only the first-change system duty changes the same cooperative raw solver's answer from false to true. Executable case checks make the audit inspectable. The results support candidate-exclusion reasoning; general installation guarantees and operational benefit remain outside the established scope.**

***Index Terms*—self-adaptive systems, reactive synthesis, continuation preservation, controller replacement, verification artifacts**

I. Introduction

Self-adaptive systems are expected to maintain required behavior as their environment changes. The monitor–analyze–plan–execute loop with shared knowledge (MAPE-K) provides a reference architecture for this process [1]. Planning may consider revised requirements or replacement controllers [2], [3], [4]. Continuing an ongoing task requires accounting for the current state and pending obligations. Before discarding a candidate, an adaptation manager asks: *can a controller meet these requirements from the retained state?* Figure 1 highlights this decision; we audit when a negative translated synthesis answer justifies exclusion.

Reactive synthesis asks for a controller that responds to environment inputs. GR(1) specifications combine initial conditions, safety rules and recurring goals for the environment and controller. Under *strict* semantics, the controller owes safety on every prefix that respects ENV initialization and safety; later unfairness cannot excuse an earlier unsafe response. Kind adds cooperation to strict winning: after every history the strategy induces, even after an environment violation, that same strategy at its reached memory must permit a safe, fair environment future. We call this property *source-kind* when it holds before translation; its full definition appears in Section V-A.

We call the requirements before translation the *source*; after an update, these are the candidate's updated requirements. Exclusion requires source-property preservation and query/objective correspondence. A raw call invokes the underlying solver; a public action also selects the translation and solver path.

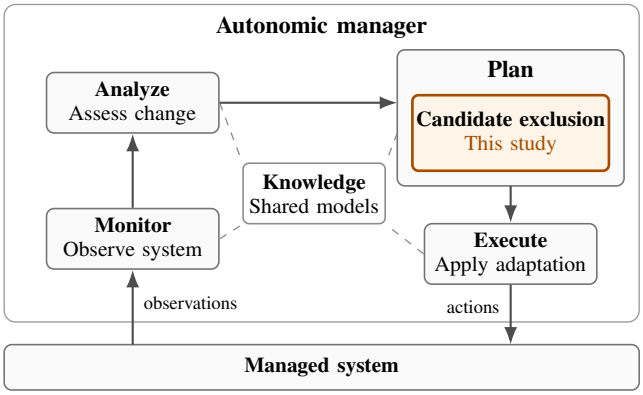


Fig. 1. Self-adaptation architecture based on Kephart and Chess [1, Fig. 2]. The highlighted planning question is the focus of this paper. The surrounding loop provides architectural context; it is not implemented here.

The helper is an environment-controlled Boolean input that can change permanently from false to true. Its first change releases ENV safety one edge before SYS safety, so the controller must still answer safely on that edge.

Patrol exposes why this timing matters. At the retained state the robot can answer every input allowed by the first helper change. Yet meeting its charging obligation lets the environment steer it into a conflict: remain at the charger and avoid a blocked checkpoint placed there. That placement is forbidden by the source but permitted by the helper. Thus being able to respond now differs from maintaining that ability while winning.

An existing complete-relation correction releases both duties on the first-change edge and preserves the whole source strict-winning region [6]. Our question is which continuations survive the original printed encoding, including exports that retain auxiliary duties; correcting the encoding does not characterize those survivors. The public Kind path inspected for the Obstacle inputs also uses a syntactic reduction and raw solver separately from that correction (Section V). We give exact preservation conditions and a local intervention on that solver query, rather than a new solver or correction. With complete safety-relation guards, the central condition is that one winning source strategy keep a safe first-change response available for as long as the helper may wait. We make three contributions:

1) *Exact preservation for exclusion.* We characterize the surviving strict-winning continuations, derive a necessary and sufficient whole-region condition, and extend the result to safety duties retained after the change. All three results have complete proofs in the main paper.
2) *Constructive and implementation-bound evidence.* A constructed ENV-only Patrol update separates responding now from maintaining that ability while winning. Vehicle separates initialization from continuation preservation. At a fixed Obstacle requirements-replacement query, releasing only the first-change SYS duty changes the same solver's answer from false to true.
3) *An exclusion contract and executable checks.* We connect query/objective correspondence and preservation to candidate exclusion. The accompanying release binds typed model snapshots to positive witnesses, loss certificates and saved evidence for Vehicle, Patrol and Obstacle B [7].

We first explain the Patrol challenge, give the basic strict result, then close Patrol's positive and negative arguments before treating retained duties. The cases then connect preservation to tool answers, before the candidate-exclusion audit and common limits.

II. A Continuation That Should Not Be Discarded

Consider a robot that repeatedly visits three checkpoints and a charger on a 5×5 grid. Charging requires a dwell period, and a request monitor records whether a charging visit is owed. These duties come from the public Patrolling specification, corpus 277 [9]. We use its unchanged source as the new model and construct an old model that additionally fixes the displayed layout and keeps all checkpoints open. Removing those old ENV restrictions leaves system requirements, types and monitors unchanged. Checkpoint positions still remain fixed from one round to the next, but a distant checkpoint may now close. This update scenario and reuse policy were constructed after diagnosis.

A. The retained entry and two safety rules

An allowed old-model step reaches entry r: the robot is at the origin $O = (0, 0)$, all checkpoints are open, charging is unavailable, the dwell counter c is zero and the charging request is pending. Its retained route phase is ready to visit checkpoint A. The checkpoints are at $(0, 4), (2, 4), (4, 4)$ and the charger at $H = (4, 0)$ (Fig. 2).

Write p for the robot position and z for charging availability; primes denote next values. A checkpoint i has blocked flag d_i and position t_i. Two SYS safety duties will conflict:

$$\begin{aligned} \text{Dwell:} \quad & (p = H \wedge c < 3) \Rightarrow p' = H, \\ \text{Avoidance:} \quad & d_i \Rightarrow p' \neq t'_i. \end{aligned} \tag{1}$$

Avoidance combines the *current* blocked flag with the *next* checkpoint position. Source-legal inputs keep positions fixed and cannot change a flag while the robot is within one cell of that checkpoint. The complete ENV justice list requires only that each of the three checkpoints reopen infinitely often. There is *no charging-availability justice*: availability is governed by the safety rule $c = 3 \Rightarrow z'$, using the current counter.

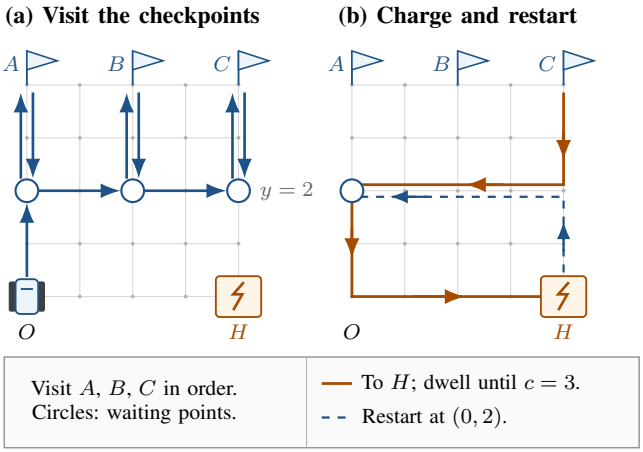


Fig. 2. Patrol's source policy on the same 5×5 grid in both panels ($O = (0, 0)$, $H = (4, 0)$; checkpoints on row four). Arrows are offset for visibility; all moves are unit cardinal. Wait two cells below each checkpoint while its next blocked flag is true, then approach, visit and retreat. Charging and restart share row two; restart at $(0, 2)$.

B. Responding now is not enough

At r, staying at O is safe even for an arbitrary typed next input: the robot is not dwelling and every current blocked flag is false. It also has a source policy that visits the checkpoints, waits for reopening when needed, and charges repeatedly; Section IV proves this using the displayed route.

The helper encoding adds an internal ENV Boolean b, initially false, that can change to true once. This is a game input, not the physical update. Table I shows the challenge: the first change releases ENV safety immediately while SYS still owes one safe response. The strict meaning of that duty is defined in Section III.

TABLE I
One edge separates the releases for the complete literal relations in Eq. (2). "Any typed" includes source-forbidden inputs. The last column uses the existing corrected guard.

Helper	Legal ENV input	SYS duty	Corrected duty
$0 \to 0$	Source-legal	Source	Source
$0 \to 1$	Any typed	Source	None
$1 \to 1$	Any typed	None	None

Table II shows how a legal source prefix reaches the point where that response fails. ENV keeps the checkpoints open and charging unavailable until the first arrival at H, then legally closes distant checkpoint A on the next round.

Clearing the request records the visit; it does not finish the dwell duty, which remains active until the counter reaches three. From x, a first-change input places the already blocked checkpoint at H: Dwell requires the robot to stay there and Avoidance forbids it. That input belongs to the translated game; moving a checkpoint is not source-legal. A different safe strategy cannot avoid the issue: never reaching H leaves the request pending forever, whereas first reaching H permits the same legal next round and conflict (Fig. 3). The proof will

TABLE II

MILESTONES OF PATROL'S LEGAL PREFIX. REACHING H MAY TAKE SEVERAL ROUNDS; THE NEXT ROW IS ONE DWELL EDGE LATER. AT x THE REQUEST HAS CLEARED BUT THE DWELL IS UNFINISHED.

State / event	p	c	Request
Retained entry r	O	0	pending
First arrival at H	H	0	pending
Next legal dwell; A closes (x)	H	1	cleared (idle)

cover every safe strategy. The question is whether a winning policy can *keep* a response available, not only whether the retained state has one now.

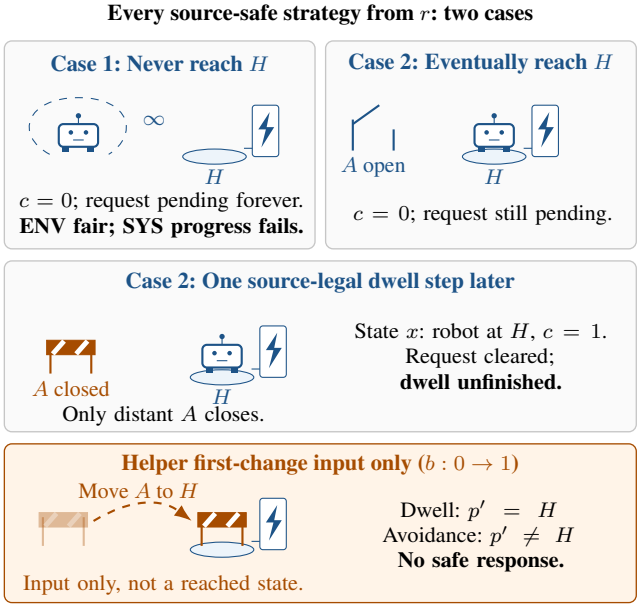


Fig. 3. A proof scenario from source-winning r for every strategy that stays safe on source-legal inputs. ENV keeps all checkpoints open and charging unavailable until first arrival at H. The orange scene moves already blocked A to H in a proposed helper input, not a physical update or an executed successor.

III. WHAT THE ENCODING PRESERVES

A. The game and the meaning of a continuation

A finite, fully observed Mealy game has nonempty typed input and response carriers E, Y and states $s = (e, y) \in D = E \times Y$, including all logical monitors. In one round ENV chooses e'; SYS observes it and chooses y', reaching $s' = (e', y')$. ENV safety $\rho_e(s, e')$ is independent of the next SYS response. SYS safety $\rho_s(s, e', y')$ includes every system constraint and monitor update. A typed input is in its declared domain; a *source-legal* input also satisfies ρ_e. Neither safety relation need be total. Invalid type encodings are outside D.

The ENV and SYS justice lists define generalized Büchi objectives A, G: each listed predicate must hold infinitely often. Under *strict* semantics [8], [5], when ENV initialization holds, SYS owes its initializer and safety on every edge whose entire ENV-safety prefix, including that edge, is legal. An actual ENV safety violation releases the safety duty on the violating edge and thereafter. On infinite plays satisfying ENV initialization and safety, SYS must satisfy $A \Rightarrow G$. Future ENV unfairness cannot excuse an earlier required safety violation. Strategies may use history and private memory and supply typed responses even after ENV violations.

Let W be the set of states from which SYS has a strict-winning source continuation. Such an *active* entry has met ENV initialization, prior ENV safety and the required SYS duties. Membership in W says neither that the state is reachable nor that it is an allowed initializer; ENV dead ends may win vacuously. Initial synthesis instead asks whether *each* allowed ENV initial value admits *some* allowed SYS response in the winning region. A retained-state query fixes both components, including the pending obligations. It cannot select a different initial response. These distinct questions will matter in Vehicle and Obstacle.

B. The first-change edge

The helper b starts false, is offered with the next ENV input and is observed before SYS responds. It can become true only once. Kind's printed clause guards relax ENV safety with the *next* helper and SYS safety with the *current* helper [5, Sec. 4.1]. When these guards cover the *complete* safety relations, including all auxiliaries, the encoded relations and objectives are

$$\begin{aligned} \rho_e^R &= (\rho_e \vee b') \wedge (\neg b \vee b'), & \rho_s^R &= \rho_s \vee b, \\ A^R &= A \wedge \mathsf{GF}\neg b, & G^R &= G. \end{aligned} \tag{2}$$

Initialization is unchanged except that ENV also fixes $b = 0$. The factor $\neg b \vee b'$ makes the change one-way. "Complete" describes the guards' coverage, not whether a safe response exists.

Table I showed the timing consequence: on the first-change edge, ENV has already been released but SYS has not. The added ENV justice $\mathsf{GF}\neg b$ becomes impossible after a permanent change to true. Strict semantics nevertheless requires the safe response on that legal edge: future unfairness does not waive present safety.

The existing correction uses $\rho_s^{R^+} = \rho_s \vee b \vee b'$ [6]. It releases the first-change duty and preserves the source strict region (Supplement L.6). The theorem below characterizes the encoded *strict game*; the public cooperative procedure is a separate object considered in Section V.

C. Winning while keeping the response option

Every typed input is legal on the first change. Define its immediate safe-response condition by

$$C(s) \iff \forall e' \in E \; \exists y' \in Y : \rho_s(s, e', y'). \tag{3}$$

There is no ENV-safety antecedent. A state in $W \cap C$ can win in the source and can answer a first change now. Those two facts need not be realized by a strategy that keeps the response option available.

Let W_C be the set of states from which there exists *one source strict-winning strategy* such that every source-legal finite play prefix induced by that strategy ends in C, including the length-zero prefix consisting of its initial state. The same strategy and its evolving private memory must both win and maintain this option. We do not replace the strategy or reset its memory at each prefix. Including the initial state also requires C at an ENV dead end. This direct definition is the game-based definition in Supplement L.6, expressed as a strategy obligation.

The proof separates what happens if the helper changes from what happens if it keeps waiting. It must account for both possibilities with one strategy.

Theorem 1 (Exact continuation preservation)**.** *Under the stated Mealy, strict and complete-relation hypotheses,* $(s, 0)$ *is strict-winning in the encoded game exactly when* $s \in W_C$*. Every state* $(s, 1)$ *is strict-winning.*

Proof. At $b = 1$, legal edges keep the helper true. All SYS safety duties are released, and every infinite legal play violates $\mathsf{GF}\neg b$. Hence every such state is winning.

For necessity, take a winning strategy from $(s, 0)$ and restrict it to source-legal histories on which the helper stays false. These histories have exactly the source safety and liveness obligations, so the restriction is source strict-winning. At every finite prefix, including the initial state, its winning residual must safely answer every typed first-change input. The residual is winning because required safety has held and the Büchi objectives are prefix-independent. Therefore every state reached by this same restricted strategy belongs to C, proving $s \in W_C$.

For sufficiency, take a source strict-winning strategy witnessing $s \in W_C$. Follow it while the helper remains false and inputs are source-legal. It maintains C, so at a first change SYS can choose a safe response to every typed input. The successor has helper true and is winning by the first paragraph. If the helper never changes, the source obligations apply unchanged. An encoded-illegal input releases strict duties. This strategy wins the encoded game. □

Corollary 2 (Whole-region preservation)**.** *The encoding preserves the entire source strict-winning region at helper false if and only if* $W \subseteq C$*.*

Proof. Necessity follows from $W_C \subseteq W \cap C$. Conversely, every state reached by a source-winning strategy along a source-legal prefix has a winning residual and therefore belongs to W. If $W \subseteq C$, the same strategy stays in C and witnesses membership in W_C. Thus every source winner is preserved. □

This is a condition for the *whole region*. It does not say that one state in $W \cap C$ survives: the strategy may need to leave C in order to win. Patrol will show exactly that. Nor does equality of initial answers imply region preservation; initialization may choose its responses entirely within the surviving region.

IV. Returning to Patrol

The same retained entry now separates the theorem's two conditions. We first show a winning source policy and an immediate safe response. We then show that *every* safe strategy either fails its recurring obligation or reaches a state with no first-change response.

A. The positive continuation

The route in Fig. 2 waits two cells below each checkpoint until its *next* blocked flag is false, approaches through row three, visits and retreats to row two. After the three visits it returns via O to H, dwells until the counter reaches three, and returns through $(4, 1)$, $(4, 2)$ and row two to the first standoff. All moves are unit cardinal steps.

The source rules justify each part of this policy. Positions of checkpoints are fixed. When the robot enters a checkpoint's one-cell neighborhood its flag is false; neighborhood stability keeps it false through the visit and retreat. The rest of the route avoids the checkpoints and obeys Dwell. ENV fairness ends each wait, so the visits recur.

For charging, the counter increments, capped at three, when the *current* position is H and is zero on every other successor. The monitor is idle, pending or failed. An idle monitor becomes pending when charging is unavailable, the current counter is not three and the robot is off the charger. A pending monitor first tests whether the counter is three, entering the absorbing failed state if so; otherwise it clears to idle when the current position is H. It remains pending otherwise. The complete SYS justice list requires infinitely many idle states and visits to each of the three checkpoints and O (five predicates). The complete source equations are in Supplements S and AB.6.

First arrival at H has counter zero. The pending request clears on the first dwell edge, before the counter reaches three, and cannot retrigger at H. The source requires charging to become available on an edge whose current counter is three. Departure therefore carries counter three, an idle monitor and available charging. On the next off-charger edge the current counter prevents retriggering while it resets. The monitor never fails and is idle infinitely often. Thus the same policy meets both safety and progress: $r \in W$.

At r, any typed next ENV input permits staying at O, keeping counter zero and the request pending. Dwell is inactive and all current blocked flags are false. Hence $C(r)$ also holds. What is missing is the ability to maintain C while winning.

B. Why every safe strategy loses that option

Let ENV keep the displayed layout, every checkpoint open and charging unavailable until the first arrival at H. This is source-legal; keeping every checkpoint open also satisfies all three ENV justices. Figure 3 divides all safe strategies into two cases.

Never reach the charger. Every safe response keeps the counter zero and the request pending: the updates inspect the current off-charger position. The resulting ENV-fair play never clears the request, violating SYS justice. Thus a source-winning strategy cannot choose this branch forever.

First reach the charger. The arrival still has counter zero and a pending request. On the next edge ENV legally closes only distant checkpoint A. Dwell keeps the robot at H; the counter

becomes one and the request clears. Call this state x. Now a typed first-change input places that already blocked checkpoint at H. Dwell requires $p' = H$ and Avoidance requires $p' \neq H$. No response exists, so $x \notin C$. The contradiction occurs *after the request clears*, while the dwell is still unfinished.

The two branches cover all safe responses, not just the route we displayed. Consequently $r \in W \cap C$ but $r \notin W_C$. Theorem 1 excludes $(r, 0)$ from the complete literal game, even though the source can continue from the same entry. The loss is caused by having to keep a response available until an adversary chooses the first change.

This proves the loss for complete literal relations. Tool exports may retain auxiliary duties after the helper changes. We extend the argument before applying it to Patrol's actual export.

C. If safety duties remain after the change

A syntactic translator may leave counter or monitor constraints unguarded. Let $U \subseteq D$ be exactly the states admitting a SYS strategy that satisfies all remaining safety duties along every legal helper-true ENV prefix. ENV dead ends are winning vacuously. Write ρ_e^{01} and ρ_s^{01} for the complete encoded ENV and SYS safety relations with $b = 0, b' = 1$. The exact safe-entry condition is

$$F(s) \iff \forall e' \in E : \ \rho_e^{01}(s, e') \Rightarrow \exists y' \in Y : \ \rho_s^{01}(s, e', y') \wedge (e', y') \in U. \tag{4}$$

Thus every legal first-change input must admit a safe successor in U. Define W_F by replacing C with F in the maintained-response definition of W_C, including its initial-state requirement.

Proposition 3 (Preservation with retained duties)**.** *Suppose the safety relations on $0 \to 0$ edges agree with the source. Apart from the added $\mathsf{GF}\neg b$ ENV assumption, the ENV and SYS justice lists agree with the source at helper false and may be arbitrary at helper true. The helper is one-way, its next value is observed before SYS responds, and ENV legality has no next-SYS dependence. Then the helper-true strict-winning region is U and the helper-false region is W_F. Whole-region preservation holds exactly when $W \subseteq F$.*

Proof. At helper true, persistence makes $\mathsf{GF}\neg b$ false on every infinite legal play. Strict winning is therefore exactly safety winning, namely U.

Restrict a winning helper-false strategy to source-legal histories where the helper stays false. Their obligations coincide with the source. Every such prefix, including the initial state, has a winning residual with its reached memory: required safety has held and the Büchi objectives are prefix-independent. Every legal first change must therefore receive a safe response into U. This same restricted strategy wins the source while maintaining F, proving membership in W_F.

Conversely, follow one source-winning strategy witnessing W_F, with its evolving memory, until a first change. Then F supplies a safe response into U; continue with a safety-winning strategy on U. Without a change, source obligations hold unchanged. An encoded-illegal input releases strict duties on that edge and thereafter. This proves both region equalities.

Finally, $W_F \subseteq W \cap F$ gives necessity for whole-region preservation. If $W \subseteq F$, every source-legal prefix of a source-winning strategy has a winning residual, so every reached state lies in $W \subseteq F$. The same strategy witnesses W_F, proving sufficiency. □

Supplement L.1 gives an example with two source states, one always-legal input value, a Boolean SYS output, and true source initial and justice predicates: source steps and first-change responses must set the next output to true. Every subsequent helper-true transition requires both current and next outputs to be false. Thus both source states win and admit an immediate safe first-change response, but that response enters a state with no safe next response, so neither state survives the encoding. Supplement L.1 gives the complete relational formulation and fixed-point construction. For complete literal relations, $U = D$ and $F = C$; these identities must be checked for an actual export.

D. The actual export loses the same entry

On the 4,800 current states with the displayed layout, symbolic and explicit calculations find **3,280 source winners and 1,104 survivors** (Supplement U.1). A completed native reduced solve matches the survivor set after the required F intersection (Supplement U.6). The fixed layout limits the counted current states, not the typed next inputs: the first-change test still permits a checkpoint to move to any typed position. The reduction enforces maintenance of F while winning (Supplement L.1). The same retained entry is excluded by the actual syntactic export, by the following relation checks.

The export retains auxiliary updates, so Proposition 3 applies. Complete typed checks establish source agreement before the change and unchanged helper-false ordered justices. At the first change every typed ENV input is legal and the source SYS duty remains. Afterwards ENV is unrestricted; the remaining counter/monitor updates permit every typed robot position with a typed auxiliary update, independently of the next ENV values. Thus $U = D$ and $F = C$, and the all-response argument excludes $(r, 0)$ from the export's strict-winning region. The original-spelling old export also loses this entry (Supplement U.1). This is an export-based strict deduction, not an observed public Kind refusal; the direct full-helper attempts remain UNKNOWN (Supplements U and AB.4).

E. What the retained-state execution shows

The recorded update reused the legal policy under the old and then new ENV model at the same retained query, preserving plant/controller objects, source fields, monitors and private phase. It completed the pending request and entered a recurring justice-satisfying cycle. The separate source-kind existence argument follows in Section V-A.

V. Evidence and Tool Answers

A. Source-kind witnesses and tool evidence

Kind adds cooperation to strict winning [5]. After *every* history induced by the strategy, including histories after ENV initialization or safety violations, the same strategy at its actually reached private memory must still permit an ENV-safe, fair future. That future reapplies no initializer, repairs no past violation and restarts no strict duty. A source strict-winning strategy with this additional property is *source-kind*.

Sections III–IV characterize strict-game preservation. We now distinguish source-kind witnesses from public-mode and raw solver answers. A *raw answer* is the cooperative four-fixed-point procedure's Boolean for a fixed initialized query, not a strict-region count or an extracted controller.

a) Patrol also has a source-kind continuation: The legal policy extends to a source-kind strategy for the same point-initialized entry and retained route phase. Its typed-total completion covers ENV initialization and safety violations while preserving every legal-history output and phase update, and the exact source counter/monitor updates throughout. From every induced history, that same strategy at its reached memory permits a safe, fair ENV future; Supplements AD.4–AD.6 give the construction and proof. The completion was not implemented or executed, and it does not waive the encoded first-change SYS duty.

B. Vehicle: can positive initialization hide lost continuations?

Yes. Both the source and syntactic-helper initialized strict queries are positive, yet **1,408 of 8,064 saved source winners are lost**. The audit covers all 33,792 typed states, finding 6,656 survivors and zero gains [14]. Zero gains agrees with $W_C \subseteq W$. Vehicle188 was outcome-selected in an exploratory sweep [12]; the whole-region audit was preregistered. Complete relation checks bind the translation to Eq. (2), and both region queries use the released GR1Game solver [13].

A retained state nevertheless exposes an immediate loss. The car is stopped in the middle lane, only that lane has an obstacle, all lights are green, and there are no pedestrians or crosswalk. Its two response rules say: if the middle lane is blocked and the left lane is blocked next, move right; if the middle lane is blocked and the right lane is blocked next, move left. The source ENV rule forbids both outer lanes from being blocked next in this situation. A first helper change permits precisely that input, so the two SYS rules require different next lanes at once (Fig. 4). No speed choice can resolve the conflict.

The source can win from this state. A source-legal next input leaves an obstacle-free outer lane. Choose it. Thereafter obey the forced lane moves; otherwise choose an obstacle-free middle lane. If the middle is blocked, stay in the current outer lane or move from the middle to an outer lane. Set speed to one when the selected lane has a green light, no pedestrian and no obstacle, and to zero otherwise. After the first response the policy selects the middle whenever that lane is free, so its recurring availability implies the recurring free-lane goal. Supplement E supplies the invariant, all speed duties and full proof.

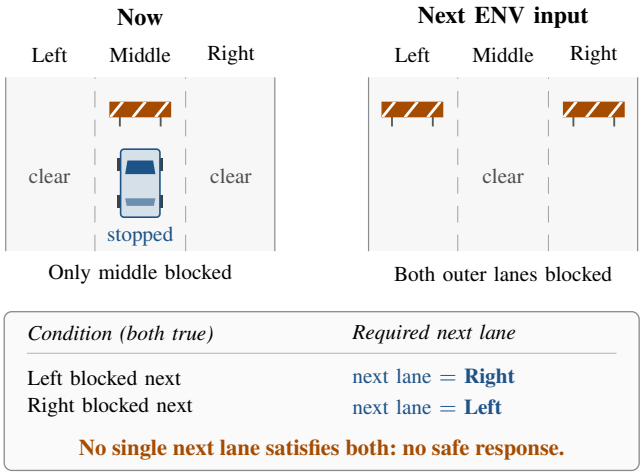


Fig. 4. Vehicle's first-change conflict. Barrier icons mark blocked lanes; their positions do not encode distance. The next ENV input is source-forbidden but helper-legal. With the car in the blocked middle lane now, the two rules require opposite next lanes simultaneously; no speed choice resolves the conflict. No completed car movement is shown.

For *each* saved source winner, including each loss, there exists an initialized Kind strategy and a compatible source-legal play reaching that target at time one. Both may depend on the target: this is neither a common controller nor forced reachability. Supplements X and AD.3 give the target-specific predecessor and all-history completion proofs. No update, public Kind query or replacement execution was performed for Vehicle.

C. Obstacle: can one first-change duty alter a solver answer?

At exact query B, releasing that duty changes the fixed raw answer from false to true. Unlike Patrol's later loss of an initially available response, Obstacle has an immediate first-change obstruction at every state. It tests the implementation consequence and the effect of releasing the duty.

The published Obstacle Evasion old/update pair [4] changes forbidden locations and recurring SYS visit goals. We consider two exact queries, each fixing its input specification and initializer. A uses the old source and its authored initializer. B uses the updated ordinary projection—the source with only the parse-time switch declaration removed—and fixes a retained point: obstacle $(3,5)$, `obsWait=true`, robot $(5,5)$. A uses obstacle $(7,7)$, `obsWait=false`, robot $(1,1)$. Each query has one ENV initial value and the specified SYS response; the full universal-ENV/existential-SYS initial obligation is retained. B's point was fixed by a seven-edge legal prefix in the old winning region before computing the new region. The old initializer is not conjoined. This is full requirements replacement, not an ENV-only update or preservation of the old pending goals.

a) Both exact queries have source-kind witnesses: Each direct-source certificate alternates between two goal phases; for B these mean visiting $(3,7)$ and $(4,8)$, with a finite nonnegative rank that is zero exactly at the current goal. A separately implemented checker enumerates every legal ENV input and

checks the supplied response for full SYS safety and closure within the certified states, strict rank decrease with unchanged phase away from the goal, and a phase switch at the goal, where no rank decrease is required. It also checks the full initial quantifiers: the accepted certificates establish a winning strategy for each exact query, not a maximal region. The ENV relation is independent of the next SYS response, total on all 6,272 typed states, and has no justice assumptions. Consequently every history of a typed-total completion of the strict witness admits an ENV-safe, fair suffix using that same strategy and reached memory. Both exact queries therefore have source-kind witnesses (Supplement AC.5–AC.7).

What a negative tool answer says about those witnesses depends on the comparison. Table III keeps the three experiments separate.

TABLE III
THREE COMPLETED OBSTACLE COMPARISONS: FIXED-STRICT REGIONS, PUBLIC-MODE ANSWERS AND THE EXACT-B RAW INTERVENTION. THE SEPARATE EXPORTED-GAME AUDIT GROUP HAS EIGHT UNKNOWNS.

Question	Procedure / change	Result
Strict survival	Fixed strict solver; source + three helper games	Release and correction recover source regions.
Public answer	Ordinary vs Kind; translation and solver change	Ordinary: true; Kind: false (A, B).
Raw effect at B	Fixed cooperative solver; duty kept → released	False → true.

b) Release and correction recover both strict source regions: Table IV gives the completed comparison under the same released strict GR1Game solver, with source requirements and types fixed. The syntactic helper loses every strict source winner. These are strict regions, not cooperative regions returned by the public Kind procedure. Source avoidance excludes the current robot from the next obstacle footprint. On a first helper change a typed input can place the footprint over that robot, regardless of its response. Export relation checks bind this obstruction to both syntactic games. First-change release removes the SYS duty only on the $0 \to 1$ quadrant; complete correction guards the whole SYS relation. Both recover the source regions state by state. These exports have $U = D$.

TABLE IV
COMPLETED FIXED STRICT-SOLVER COMPARISON: WINNING-STATE COUNTS. THE UPDATE REGION HAS NO INITIALIZER; EXACT POINT QUERY B IS SEPARATE.

Game	Old	Update
Source	5,016	5,015
Syntactic helper, $b = 0$	0	0
First-change release, $b = 0$	5,016	5,015
Complete correction, $b = 0$	5,016	5,015

The old query retains its authored initializer. B's point lies in the recovered update region and its singleton initial quantifiers give the corresponding strict conclusion. This fixed-strict comparison completed two source and six translated native queries (Supplement AC.2–AC.4).

c) Public ordinary mode is positive and Kind mode negative: For each of A and B, the public ordinary-GR(1) action returned true and the Kind action false, once per mode under the same release and backend settings. Mode selection changes *both* translation and solver, so this comparison alone does not isolate a cause. Code and console records identify a static syntactic-reduction/raw-solver path, separate from the complete correction. A was already negative before the update (Supplement AE).

d) Releasing one duty changes the raw answer at B: At exact B, two fresh processes called the same cooperative four-fixed-point raw solver and completed normally: unchanged returned **false**; first-change release returned **true**. Input and initializer checks bind the source certificate and separate ENV-totality result to both raw arms' source query, with the helper initially false. With translation/backend settings fixed, typed comparisons verify all other relations, domains, initializers and ordered justices unchanged; quadrant differences verify complete SYS release only on typed $0 \to 1$, with typed $1 \to 1$ already true. Supplement AG records this local intervention and execution protocol; AF retains the separate earlier false/UNKNOWN protocol.

If used as a continuation gate at B, the public Kind negative would reject the source-kind witness above. The raw pair establishes a local intervention effect; exclusion still requires the query-and-objective correspondence below.

D. Cohort: which losses are ruled out by the certified sample?

The cohort asks where loss can be ruled out, rather than how often it occurs. Loss of all-history source-kind winners is excluded in 58 models for the *literal strict target*, although ten models have strict-state losses. The remaining model's exact cooperative-loss intersection is UNKNOWN. Table V shows the complete 49/9/1 partition.

Here *legal-prefix cooperation* means that after every source-legal prefix induced by one strict-winning strategy, that same strategy at its reached memory permits an ENV-safe, fair continuation. All-history source-kind winners form a subset, because they also require cooperation after violations.

TABLE V
STOPPED, COMPLETION-AND-CERTIFICATE-CONDITIONED COHORT. THE PRESERVATION TARGET IS THE COMPLETE LITERAL STRICT ENCODING.

Models	Established outcome
49	Entire strict source-winning region preserved.
9	Strict-state losses occur, but no legal-prefix cooperative source winner is lost.
1	Strict loss is known; the exact cooperative-loss intersection remains UNKNOWN (corpus159, RoboticArm).

No cooperative-initialization loss is confirmed. Of 425 public units, 70 completed and 59 remained after eleven certification exclusions, without outcome filtering within the eligible set. All three cases lie outside this stopped cohort. Completion and certification conditioning preclude an incidence or rarity estimate for operational Kind/update states or public actions. Supplement AB.5 records case selection and Patrol277's certificate exclusion, which its later proof does not undo.

VI. Auditing Candidate Exclusion

A. An exclusion contract

In replacement planning, a negative answer can exclude a source continuation only when the query, translation and answering procedure support that inference. The result gives an implementer's reasoning contract with three steps.

a) Fix the query and objective: Decide whether the question is about a retained state or about all allowed initial ENV values and their selectable SYS responses. Also distinguish strict winning from a cooperation requirement such as Kind. For a retained entry, fields and pending monitors are part of the state. Initial realizability cannot substitute for this query, and installing a particular controller additionally requires compatible private memory.

b) Establish preservation for that query: Check that the actual translation has the required typed relations, auxiliary updates, initializers and ordered justices. For a fixed retained state s, preservation means $s \in W \Rightarrow s \in W_C$, or $s \in W \Rightarrow s \in W_F$ under retained duties (Theorem 1 and Proposition 3). A strategy may leave a queried set and must still maintain its response option; immediate responses alone are insufficient.

With unchanged initializers and initial $b = 0$, initialized strict synthesis instead compares the $\forall$ENV$\exists$SYS initial decisions. For an initially realizable source, the decision is preserved exactly when every allowed ENV initial value admits at least one allowed SYS response in W_C (or W_F). It need not preserve every winning initial response. Source initial unrealizability is already preserved by $W_C, W_F \subseteq W$.

Whole-region preservation holds exactly when $W \subseteq C$ or $W \subseteq F$. This is sufficient, but generally not necessary, for initial-decision preservation. Checking these conditions can require game solving; no inexpensive online test is implied.

c) Establish what the negative answer rules out: With the corresponding preservation obligation, a sound negative *strict* answer rules out the source property queried: winning from the fixed retained state, or initialized realizability. A public Kind procedure has a different intended objective; its negative answer needs its own correspondence to that source objective. Our strict theorem does not supply it.

To rule out source-kind existence, it suffices that such existence imply a positive encoded strict answer for the same retained state or initial quantifiers. Preserving all strict source winners is therefore conservative. Missing preservation or correspondence means exclusion lacks support; that is neither a positive source result nor a solver UNKNOWN.

Patrol gives the concrete obstruction: a positive source query lies outside W_F, so its encoded strict negative cannot exclude it. Obstacle adds a source-kind positive premise and negative public answer, with a local answer-changing intervention at B. That intervention does not replace the query-and-objective correspondence required by the contract.

B. Inspecting the accompanying artifact

The accompanying release [7] provides three checks. Vehicle checks saved loss membership and target-specific predecessors. Patrol checks the positive policy, all-response countercertificate and typed export premises. Obstacle B checks the shared query and sole SYS release behind the positive witness and raw false/true pair. The supplement reader guide and artifact reading map give the detailed tasks and evidence locations.

These checks reuse saved native results and case certificates; file integrity and document source closure remain separate from scientific replay.

VII. Scope and Relation to Prior Work

A. Limits shared by the cases

The cases and conditioned cohort do not estimate operational incidence. Positive certificates establish existence, not maximality, while saved-region comparisons share native-solver, exporter and type-metadata dependencies; the case checks independently validate neither saved-region maximality nor the compiler. The exact-B intervention does not reconstruct a historical GUI game or establish a corrected public pipeline, controller extraction or transfer.

The Patrol execution discussed here reused its legal policy without gate mediation or new controller installation. Supplements U.4 and U.9 separately record bounded software-adapter admission and installation; they do not implement the exclusion contract above. The off-assumption completion is not a physical recovery demonstration. Existence does not transfer arbitrary old-controller memory. We assume prepared candidate requirements and a valid hold of the retained entry; a changed state needs a verified update bridge or revalidation before installation. Inspection cost, benefit over direct source solving, gate/installation benefit and performance advantages remain open.

B. Relation to prior work

Adaptation methods construct choices: multi-tier control prepares alternatives, exploration discovers environment behavior, and weakening, degradation and recovery change requirements [15], [16], [17], [18]. Interpolation and permissive assumptions refine admissible ENV behavior [19], [20]. Our characterization addresses a different step: when a translated negative answer justifies excluding a source continuation from those choices.

Dynamic-update synthesis constructs controller bridges [21], [4]; runtime verification retains monitors across requirement changes [22]. These works explain why reached states and pending obligations matter. Preservation is a condition to establish before using an encoded negative to reject a candidate for such a bridge or installation.

Cooperative synthesis, well-separation and best-effort GR(1) address assumption handling and fairness opportunities [10], [11], [23], [24]. Kind adds an all-history condition [5]; we establish that positive source premise separately from strict-region preservation. Extraction has additional hypotheses: IROS constrains middle-fixed-point choices [25, author appendix, p. 9], while acceleration assumes language inclusion [26, Secs. 2.3, 4.1]. We do not derive those guarantees from the preservation theorem.

Earlier GR(1) work studies preprocessing false negatives, completeness conditions and a correction [27], distinguishing implication and strict realizability. Our source and encoded games both use strict semantics; the existing helper correction and our preservation contribution are distinguished in Section I.

VIII. Conclusion

After an environment-model change, using an encoded negative to exclude a continuation requires both preservation and query/objective correspondence. For the one-way helper encoding, a safe response now does not ensure preservation: one source-winning strategy must maintain the first-change response option. With retained duties, that response must enter a region that remains safe. Patrol demonstrates the distinction through a positive policy and an argument covering all safe strategies. Vehicle and Obstacle connect preservation to initialization and tool queries, including the controlled false-to-true result at exact B. The characterization and executable case checks make candidate-exclusion reasoning in replacement planning explicit and inspectable.

Acknowledgments

We used OpenAI Codex [28] and Anthropic Claude Code [29] to refine this research through experiment design, derivation, implementation, execution, auditing and analysis. These internal AI reviews are neither human review nor proof-assistant verification. The public artifact [7] records these checks' workflow and evidence scope.

References

[1] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, Jan. 2003. doi: 10.1109/MC.2003.1160055.

[2] T. Buckworth, D. Alrajeh, J. Kramer, and S. Uchitel, "Adapting specifications for reactive controllers," in *SEAMS*, 2023, pp. 1–12. doi: 10.1109/SEAMS59076.2023.00012.

[3] T.-A. Georgescu, D. Alrajeh, and S. Uchitel, "Learning to adapt GR(1) specifications through degradation," arXiv:2609.12231v1, Sept. 2026. https://arxiv.org/abs/2609.12231v1.

[4] G. Amram, S. Maoz, I. Segall, and M. Yossef, "Dynamic update for synthesized GR(1) controllers," in *ICSE*, 2022, pp. 786–797. doi: 10.1145/3510003.3510054.

[5] A. Gorenstein, S. Maoz, and J. O. Ringert, "Kind controllers and fast heuristics for non-well-separated GR(1) specifications," in *ICSE*, 2024. doi: 10.1145/3597503.3608131.

[6] Software Modeling Lab, "Spectra Tools: Released Game-Model Bundle," version 1.0.0.202402072123, 2024. Bundled source: `PitermanSemanticReduction`. https://smlab.cs.tau.ac.il/syntech/spectra/tools/update/plugins/tau.smlab.syntech.gamemodel_1.0.0.202402072123.jar.

[7] Anonymous artifact contributors, *Controller Replacement Audit: Accompanying Artifact and Paper Reading Map*, v71-r10, Oct. 2026. https://github.com/research-artifact-archive/seams2027/blob/controller-replacement-audit-20261026-v10/paper/README.md.

[8] R. Bloem, B. Jobstmann, N. Piterman, A. Pnueli, and Y. Sa'ar, "Synthesis of Reactive(1) Designs," *J. Comput. Syst. Sci.*, vol. 78, no. 3, pp. 911–938, 2012. doi: 10.1016/j.jcss.2011.08.007.

[9] A. Gorenstein, S. Maoz, and J. O. Ringert, "Artifact for ICSE'24 paper: Kind Controllers and Fast Heuristics for Non-Well-Separated GR(1) Specifications," Zenodo, v3, 2024. doi: 10.5281/zenodo.10553249.

[10] R. Bloem, R. Ehlers, and R. Könighofer, "Cooperative reactive synthesis," in *ATVA*, 2015, pp. 394–410. doi: 10.1007/978-3-319-24953-7_29.

[11] R. Majumdar, N. Piterman, and A.-K. Schmuck, "Environmentally-friendly GR(1) synthesis," in *TACAS*, 2019, pp. 229–246. doi: 10.1007/978-3-030-17465-1_13. Extended proofs: https://arxiv.org/abs/1902.05629.

[12] Anonymous, *Vehicle188 Native Strict Evidence*, version vehicle188-native-evidence-20261024-v1, Oct. 2026. https://github.com/research-artifact-archive/seams2027/blob/vehicle188-native-evidence-20261024-v1/VEHICLE188_NATIVE_EVIDENCE.md.

[13] S. Maoz and J. O. Ringert, "Spectra: a specification language for reactive systems," *Software and Systems Modeling*, vol. 20, pp. 1553–1586, 2021. doi: 10.1007/s10270-021-00868-z.

[14] Anonymous artifact contributors, *Vehicle188 whole-region saved evidence*, version vehicle188-whole-region-evidence-20261024-v1, Oct. 2026. https://github.com/research-artifact-archive/seams2027/blob/vehicle188-whole-region-evidence-20261024-v1/VEHICLE188_WHOLE_REGION_EVIDENCE.md.

[15] N. D'Ippolito, V. Braberman, J. Kramer, J. Magee, D. Sykes, and S. Uchitel, "Hope for the best, prepare for the worst: Multi-tier control for adaptive systems," in *ICSE*, 2014, pp. 688–699. doi: 10.1145/2568225.2568264.

[16] M. Keegan, V. Braberman, N. D'Ippolito, N. Piterman, and S. Uchitel, "Control and discovery of environment behaviour," *IEEE Transactions on Software Engineering*, vol. 48, no. 6, pp. 1965–1978, 2022. doi: 10.1109/TSE.2020.3044532.

[17] S. Chu, E. Shedden, C. Zhang, R. Meira-Góes, G. A. Moreno, D. Garlan, and E. Kang, "Runtime resolution of feature interactions through adaptive requirement weakening," in *SEAMS*, 2023, pp. 115–125. doi: 10.1109/SEAMS59076.2023.00025.

[18] S. Chu, J. Koe, D. Garlan, and E. Kang, "Integrating graceful degradation and recovery through requirement-driven adaptation," in *SEAMS*, 2024, pp. 122–132. doi: 10.1145/3643915.3644090.

[19] D. G. Cavezza and D. Alrajeh, "Interpolation-based GR(1) assumptions refinement," in *TACAS*, 2017, pp. 281–297. doi: 10.1007/978-3-662-54577-5_16.

[20] A. Anand, K. Mallik, S. P. Nayak, and A.-K. Schmuck, "Computing adequately permissive assumptions for synthesis," in *TACAS*, 2023, pp. 211–228. doi: 10.1007/978-3-031-30820-8_15.

[21] L. Nahabedian, V. Braberman, N. D'Ippolito, S. Honiden, J. Kramer, K. Tei, and S. Uchitel, "Dynamic update of discrete event controllers," *IEEE Transactions on Software Engineering*, vol. 46, no. 11, pp. 1220–1240, 2020. doi: 10.1109/TSE.2018.2876843.

[22] M. Carwehl, T. Vogel, G. N. Rodrigues, and L. Grunske, "Runtime verification of self-adaptive systems with changing requirements," in *SEAMS*, 2023, pp. 104–114. doi: 10.1109/SEAMS59076.2023.00024.

[23] S. Maoz and J. O. Ringert, "On well-separation of GR(1) specifications," in *FSE*, 2016, pp. 362–372. doi: 10.1145/2950290.2950300.

[24] S. Liu, Y. Zhang, C. Hu, and W. Dong, "Best-effort GR(1) synthesis: Exploiting environmental cooperation under unrealizability," in *ASE*, 2026, pp. 1806–1818. doi: 10.1145/3832783.3837439.

[25] R. Ehlers, R. Könighofer, and R. Bloem, "Synthesizing cooperative reactive mission plans," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015, pp. 3478–3485. doi: 10.1109/IROS.2015.7353862. Author version with appendix: https://www.ruediger-ehlers.de/papers/iros2015.pdf.

[26] S. Liu and W. Dong, "Accelerating kind realizability: A multi-stage incremental realizability checking framework," in *FM*, 2026, pp. 240–258. doi: 10.1007/978-3-032-26204-2_13.

[27] U. Klein and A. Pnueli, "Revisiting synthesis of GR(1) specifications," in *Hardware and Software: Verification and Testing (HVC 2010)*, LNCS 6504, 2011, pp. 161–181. doi: 10.1007/978-3-642-19583-9_16.

[28] OpenAI, *Codex*, 2026. Accessed Oct. 23, 2026. https://openai.com/codex/.

[29] Anthropic, *Claude Code*, 2026. Accessed Oct. 26, 2026. https://claude.com/product/claude-code.